

AI – ASSISTED FULL STACK WEB DEVELOPMENT

Week 6 — Authentication & Security

1. Overview of the Week

This week you add users to your app. You will implement registration and login with proper password hashing, generate and validate JWT tokens for API authentication, add OAuth login (Google or GitHub) for one-click sign-in, and apply basic security practices like input validation and rate limiting. By the end of this week, your app has real, secure user accounts.

2. What You Will Do This Week

- **User Registration with bcrypt:**
 - Create a POST `/api/auth/register` endpoint that accepts email and password.
 - Hash the password using bcrypt (10–12 salt rounds) before storing in the database.
 - Never return or store the plain password anywhere.
 - Handle duplicate email errors with a clean 400 response.
- **User Login with JWT:**
 - Create POST `/api/auth/login` that validates credentials and returns a signed JWT on success.
 - Use the `jsonwebtoken` library. Sign with a secret stored in `.env` as `JWT_SECRET`.
 - Set token expiry to a reasonable duration (e.g., 7 days).
- **Protected API Routes:**
 - Create a middleware (`requireAuth` or `verifyToken`) that validates the JWT from the Authorization: Bearer <token> header.
 - Apply this middleware to endpoints that should only be accessible to logged-in users.
- **Protected Frontend Routes:**
 - Store the JWT in `localStorage` (or HTTP-only cookie for better security).
 - Create a route guard component in React or Next.js that redirects unauthenticated users to `/login`.
- **OAuth Login (Google or GitHub):**
 - **Next.js path:** use `NextAuth.js` — npm install `next-auth`, follow the official guide to add Google or GitHub provider.
 - **Custom Express path:** use `Passport.js` with `passport-google-oauth20` or `passport-github2`.
- **Security Basics:**
 - Input validation: use `zod` or `express-validator` to validate request bodies.

AI – ASSISTED FULL STACK WEB DEVELOPMENT

- Rate limiting: install express-rate-limit and apply to auth endpoints (e.g., max 5 login attempts per 15 min).
- CORS: configure to allow only your frontend origin in production.

3. Learning Outcomes for the Week

Week Outcome	Mapped to CLO
Implement password-based registration and login with bcrypt hashing	CLO-4
Generate and validate JWT tokens for authenticated API access	CLO-4
Build protected frontend routes with redirect for unauthenticated users	CLO-4
Add OAuth login using a third-party provider	CLO-4
Apply security basics — input validation, rate limiting, CORS	CLO-4

4. Resources

4.1 YouTube Videos

#	Title / Content	Channel & Link
1	JWT Authentication Tutorial — Node.js + Express	Web Dev Simplified — https://www.youtube.com/watch?v=mbsmsi7l3r4
2	Node.js and bcrypt Password Hashing	Web Dev Simplified — https://www.youtube.com/watch?v=AzA_LTDoFqY
3	NextAuth.js — Complete Tutorial	Hamed Bahram — https://www.youtube.com/watch?v=md65iBX5Gxg
4	OAuth Explained in 100 Seconds	Fireship — https://www.youtube.com/watch?v=996OiexHze0
5	JWT in 100 Seconds	Fireship — https://www.youtube.com/watch?v=UBUNrFtufWo
6	Build a Full Auth System with MERN	JavaScript Mastery — https://www.youtube.com/watch?v=2-lanDWJi3k
7	Google OAuth with Passport.js	Web Dev Simplified — https://www.youtube.com/watch?v=Q0a0594tOrc
8	Rate Limiting in Express	Traversy Media — https://www.youtube.com/watch?v=uOLQWQGQz-Xg

AI – ASSISTED FULL STACK WEB DEVELOPMENT

9	Protected Routes in React Router	The Net Ninja — https://www.youtube.com/watch?v=2k8NleFjG7I
10	Zod Schema Validation Tutorial	Web Dev Simplified — https://www.youtube.com/watch?v=L6BE-U3oy80

4.2 Blogs and Articles

#	Title	Source & Link
1	NextAuth.js Documentation	NextAuth — https://next-auth.js.org/getting-started/introduction
2	JSON Web Tokens Introduction	jwt.io — https://jwt.io/introduction
3	bcrypt.js on npm	npm Docs — https://www.npmjs.com/package/bcrypt
4	OAuth 2.0 Simplified	aaronparecki.com — https://aaronparecki.com/oauth-2-simplified
5	Node.js Authentication Tutorial	DigitalOcean Community — https://www.digitalocean.com/community/tutorials/nodejs-jwt-expressjs
6	Password Storage Best Practices	OWASP Cheat Sheet Series — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
7	Rate Limiting with express-rate-limit	npm Package — https://www.npmjs.com/package/express-rate-limit
8	Zod — TypeScript-first Schema Validation	Zod Docs — https://zod.dev
9	Security Best Practices for Node.js	Express Security Guide — https://expressjs.com/en/advanced/best-practice-security.html
10	OWASP Top 10 Security Risks	OWASP — https://owasp.org/www-project-top-ten

AI – ASSISTED FULL STACK WEB DEVELOPMENT

4.3 MOOCs / Courses

#	Course	Platform & Link
1	Authentication and Authorization	Coursera (Meta Back-End Developer) — https://www.coursera.org/professional-certificates/meta-back-end-developer
2	Web Security Fundamentals	edX — https://www.edx.org/learn/cybersecurity
3	Information Security — NPTEL	SWAYAM / NPTEL (Free) — https://swayam.gov.in/explorer?searchText=information+security
4	Full Stack Open — Part 4 (Auth Module)	University of Helsinki (Free) — https://fullstackopen.com/en/part4

5. Deliverables

Deliverable 1: Working Auth System in GitHub Repo

- Register, login, and logout all functional end-to-end (frontend + backend + database).
- JWT-based protected routes — attempting to access a protected API without a token returns 401.
- At least 2 protected frontend pages that redirect unauthenticated users to /login.
- OAuth login working — user can click "Sign in with Google" (or GitHub), complete the flow, and land back on the app logged in.
- Rate limiting active on /api/auth/login and /api/auth/register.
- Input validation on all auth endpoints.

Deliverable 2: Auth Flow Test Screenshots (PDF)

- Screenshots showing:
 - Registration form and success response
 - Login form and success (JWT returned in Network tab)
 - Attempting to access a protected route without login (redirected to /login)
 - Successful OAuth login flow (at least 2 screenshots: OAuth consent screen → logged-in state)
 - Rate limit error when hitting login repeatedly (429 response)
- File name: W6_AuthFlowScreenshots_[InternID].pdf

Deliverable 3: Postman Auth Collection

- Updated Postman collection including:

AI – ASSISTED FULL STACK WEB DEVELOPMENT

- Register user request
- Login request (saves JWT as an environment variable)
- At least 2 protected endpoint requests using the saved JWT
- Export as JSON.
- File name: W6_AuthAPICollection_[InternID].json

6. Submission Instruction

- All submissions must be completed through the designated Google Form shared for Week 6 deliverables. **Submissions through any other medium will not be considered.**
- In case of multiple files, it is mandatory to **compress all files into a single .zip folder** before uploading. Only one consolidated file should be submitted in the file upload section of the form.
- If an intern is unable to meet the submission deadline due to a genuine emergency, a formal email must be sent to **internbti@geu.ac.in** by Sunday of the same week. Requests without valid justification or **submissions after the deadline** will not be considered.